

EDUGENIE

Google Gemini Powered Learning Assistant

PROJECT DOCUMENTATION

Field	Detail
Project Type	Generative AI / Web Application
Backend	Python + FastAPI
Frontend	HTML + CSS + JavaScript
AI Service	Google Gemini API
Team Leader	Roshan P
Team ID	SWTID-2026-4072
Academic Year	2026

Table of Contents

1. Project Overview
 2. Brainstorming & Ideation Phase
 3. Requirement Analysis Phase
 4. Project Design Phase
 5. Project Planning Phase
 6. Project Development Phase
 7. Project Testing Phase
 8. Project Documentation Phase
 9. Project Demonstration Phase
 10. Conclusion
 11. Future Enhancements
- Appendix A – Project Structure
- Appendix B – Source Code

1. Project Overview

1.1 Project Title

EduGenie – Google Gemini Powered Learning Assistant

1.2 Introduction

EduGenie is a Generative AI-based educational web application designed to help students learn more effectively using artificial intelligence.

The system combines a simple web interface with a FastAPI backend and Google Gemini to provide different learning functions from one application. The main features of EduGenie are:

- Question and Answer
- Simplified concept explanation
- Quiz generation
- Text summarization
- Personalized learning recommendations

1.3 Objective

The main objective of EduGenie is to create an accessible AI-powered learning assistant that allows students to interact with educational content through a simple browser interface. Instead of using separate tools for different learning activities, EduGenie combines these activities into a single application.

1.4 Main Features

Question Answering

Students can enter an academic question and receive a clear AI-generated answer.

Concept Explanation

Students can enter a difficult topic and receive a beginner-friendly explanation using simple language and examples.

Quiz Generation

EduGenie generates exactly three multiple-choice questions with four options for each question.

Summarization

Students can enter a long educational passage and receive a shorter revision-friendly summary.

Learning Recommendations

EduGenie can generate a learning path that progresses from beginner-level concepts towards advanced concepts.

2. Brainstorming & Ideation Phase

2.1 Problem Identification

Students often need different kinds of assistance while studying. For example, a student may:

- Have a question about a topic.
- Find a particular concept difficult to understand.
- Need a short summary before an examination.
- Want questions to test their knowledge.
- Need guidance about what to learn next.

Normally, these activities may require different websites or applications. Therefore, the idea was developed to create a single educational assistant capable of handling multiple learning requirements.

2.2 Brainstorming Idea

The initial concept was:

“Can Generative AI be used to create one simple learning assistant that can answer questions, explain concepts, create quizzes, summarize study material and guide students through a learning path?”

This idea resulted in the development of EduGenie.

2.3 Proposed Solution

EduGenie provides one browser-based interface where the user:

- Selects a learning task.
- Enters a question, topic or educational text.
- Submits the request.
- The FastAPI backend processes the request.
- The appropriate AI module generates the response.
- The response is displayed in the browser.

2.4 Identified Use Cases

- Ask an academic question.
- Explain a difficult concept.
- Generate a quiz.
- Summarize educational content.
- Generate a learning path.

2.5 Expected Benefits

- One platform for multiple learning activities.
- Easy interaction through a web browser.
- Beginner-friendly explanations.
- AI-generated revision material.
- Structured learning recommendations.

3. Requirement Analysis Phase

3.1 Functional Requirements

The system should provide:

- A browser-based user interface.
- A task-selection dropdown.
- A text input area.
- A submit button.
- Question-answer functionality.
- Concept explanation functionality.
- Quiz generation functionality.
- Summary generation functionality.
- Learning recommendation functionality.
- Dynamic display of AI-generated results.
- Input validation.
- Error handling.

The project reference specifies FastAPI endpoints for /qa, /explain, /quiz, /summarize, and /learn/recommendations.

3.2 Non-Functional Requirements

- Have a simple user interface.
- Be responsive.
- Use modular Python code.
- Keep API credentials outside the source code.
- Be easy to run locally.
- Separate frontend and backend responsibilities.
- Provide understandable output.

3.3 Hardware Requirements

A basic computer capable of running Python and a modern web browser is sufficient for local development. Recommended:

- Modern laptop/desktop
- Minimum 4 GB RAM
- Internet connection
- Modern browser

3.4 Software Requirements

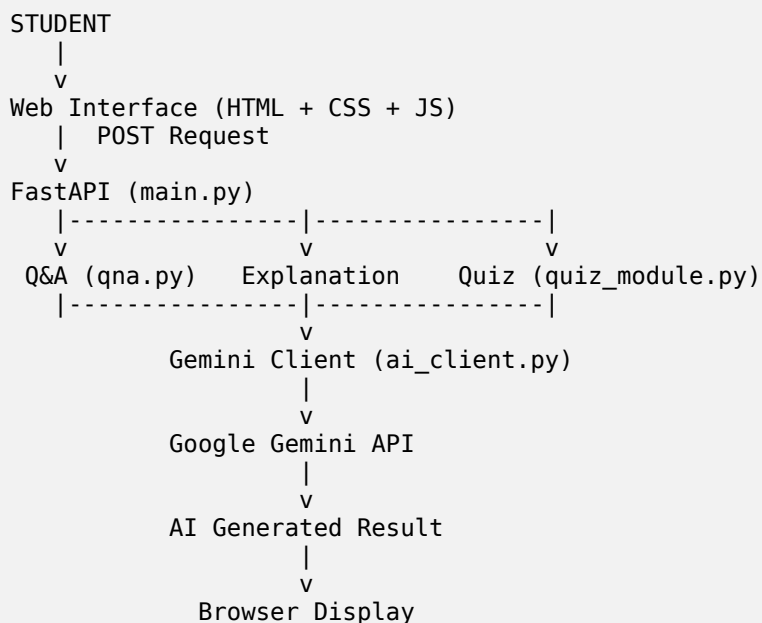
Requirement	Technology
Programming Language	Python 3.10+

Requirement	Technology
Backend	FastAPI
Server	Uvicorn
Frontend	HTML, CSS, JavaScript
AI	Google Gemini API
Templating	Jinja2
Package Manager	pip
Testing	Pytest
Environment Management	Python virtual environment

4. Project Design Phase

4.1 System Architecture

EduGenie follows a modular client-server architecture. The browser sends a selected task and input to FastAPI, which routes it to the appropriate module and uses the shared Gemini client where applicable.



4.2 Backend Design

The backend is implemented using FastAPI. The main backend file is main.py. It is responsible for:

- Creating the FastAPI application.
- Serving the frontend.
- Defining API endpoints.
- Receiving user requests.
- Routing requests to the correct feature module.

4.3 AI Client Design

The file `ai_client.py` provides a shared interface to Google Gemini. This avoids repeating Gemini API configuration in every module. The client provides functions for:

- `generate_text()`
- `generate_json()`

The API key is loaded from the environment rather than being directly written into the source code.

4.4 Feature Modules

EduGenie uses separate modules for different learning functions:

- `qna.py`
- `explanation_module.py`
- `quiz_module.py`
- `summary_module.py`
- `learning_path.py`

This modular structure makes the project easier to maintain and test.

4.5 Data Validation

The file `schemas.py` contains Pydantic models used to validate requests and generated data. For example, the quiz structure validates:

- Three questions.
- Four options per question.
- Correct answer.
- Explanation.

5. Project Planning Phase

5.1 Development Plan

The project was planned in multiple stages.

Stage 1 – Environment Setup

- Install Python.
- Create virtual environment.
- Install dependencies.
- Configure Gemini API key.

Stage 2 – Backend Development

- Create FastAPI application.
- Create API routes.
- Create AI client.
- Create feature modules.

Stage 3 – Frontend Development

- Create HTML interface.
- Add CSS styling.
- Add JavaScript interaction.
- Connect frontend to backend.

Stage 4 – AI Integration

- Integrate Google Gemini.
- Create prompts.
- Process AI responses.
- Validate structured responses.

Stage 5 – Testing

- Test API endpoints.
- Test input validation.
- Test quiz output.
- Test application through browser.

Stage 6 – Demonstration

- Run the application locally.
- Demonstrate all major features.
- Capture screenshots.
- Prepare project documentation.

The supplied project material also describes development milestones covering backend API development, frontend integration, local execution and functional testing.

6. Project Development Phase

6.1 Development Environment

The project was developed using Visual Studio Code, Python, FastAPI, Uvicorn, Google Gemini API, HTML, CSS and JavaScript.

The application was executed locally using:

```
uvicorn main:app --reload
```

The application can then be accessed at:

```
http://127.0.0.1:8000
```

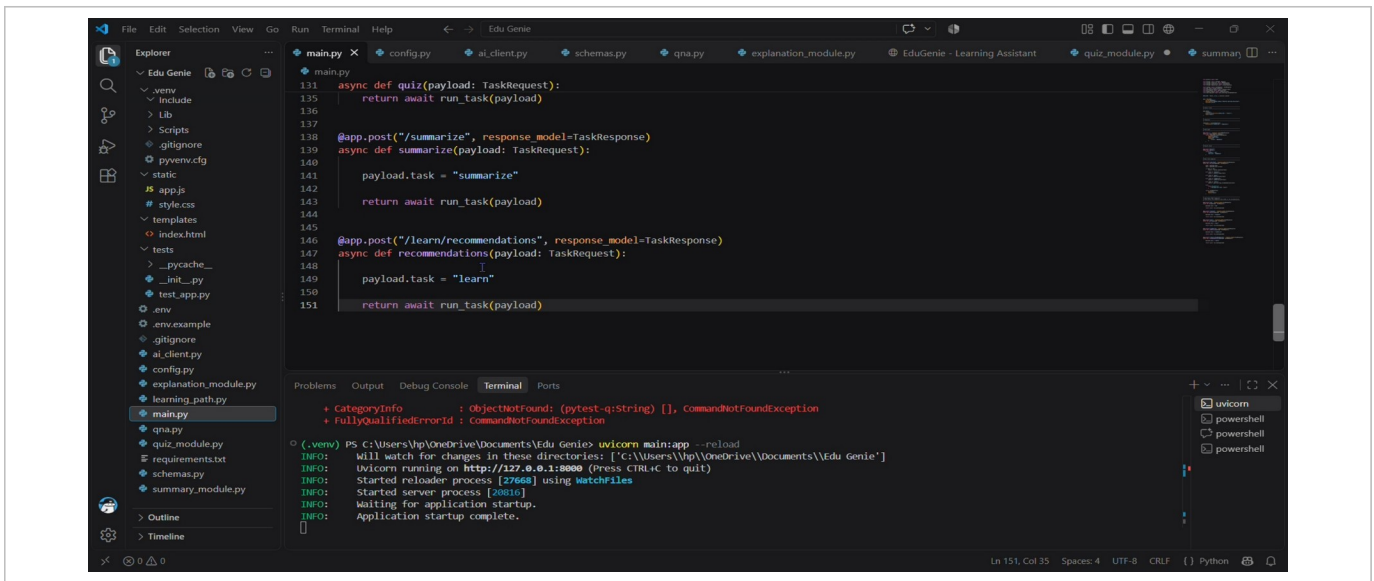


Figure 1: EduGenie FastAPI server running successfully in Visual Studio Code.

6.2 Project Folder Structure

```

EduGenie
├── main.py
├── config.py
├── ai_client.py
├── schemas.py
├── qna.py
├── explanation_module.py
├── quiz_module.py
├── summary_module.py
├── learning_path.py
├── requirements.txt
├── .env
├── .env.example
├── .gitignore
├── templates/
│   └── index.html
├── static/
│   ├── style.css
│   └── app.js
└── tests/
    ├── __init__.py
    └── test_app.py

```

6.3 main.py

main.py is the main entry point of the application. It:

- Creates the FastAPI application.
- Mounts static files.
- Loads the HTML template.
- Defines the main API route.
- Routes tasks to their respective modules.

The main tasks are: qa, explain, quiz, summarize, learn.

6.4 config.py

The configuration module loads environment variables. Important configuration values include:

- GEMINI_API_KEY
- GEMINI_MODEL
- USE_LOCAL_EXPLAINER
- LOCAL_EXPLAINER_MODEL
- MAX_INPUT_CHARS

The Gemini API key is intentionally stored in .env rather than directly in the source code.

6.5 ai_client.py

This module connects EduGenie to Google Gemini. The application uses the Gemini client to generate normal text responses and structured JSON responses. The shared AI client allows the individual feature modules to focus on their own functionality.

6.6 qna.py

The Q&A module receives the student's question and creates a prompt for Gemini. The prompt instructs the AI to:

- Answer accurately.
- Use simple language.
- Explain difficult terminology.
- Give examples where useful.
- Keep the response suitable for students.

6.7 explanation_module.py

This module is responsible for explaining difficult concepts. The current implementation can optionally use MBZUAI/LaMini-Flan-T5-783M when local explanation mode is enabled. If the local model is not used or fails, the module falls back to Gemini. The explanation prompt requests:

- Plain language.
- Beginner-friendly explanations.
- Simple analogy or example.
- Key takeaways.

6.8 quiz_module.py

The quiz module generates structured multiple-choice questions. The required output is:

- 3 questions
- 4 options per question
- 1 correct answer
- Explanation for each answer

The module also validates that the correct answer actually exists among the available options.

6.9 summary_module.py

The summary module converts a long educational passage into a concise revision-friendly format. The prompt instructs the AI to retain important facts, definitions, important relationships and conclusions. Unnecessary repetition is removed.

6.10 learning_path.py

The learning path module generates a structured study plan. The generated plan can contain:

- Topic.
- Learning level.
- Subtopics.
- Estimated time.
- Objectives.
- Resources.
- Study tips.

The project requirement specifies progression from beginner to advanced concepts and recommends useful resources.

6.11 Frontend Development

The frontend consists of templates/index.html, static/style.css and static/app.js.

index.html

Provides the user interface. The interface contains: EduGenie title, task dropdown, text input, submit button, result section.

style.css

Controls layout, typography, buttons, input fields, result container and responsive appearance.

app.js

Handles form submission, sending POST requests, receiving the backend response and displaying the result dynamically.

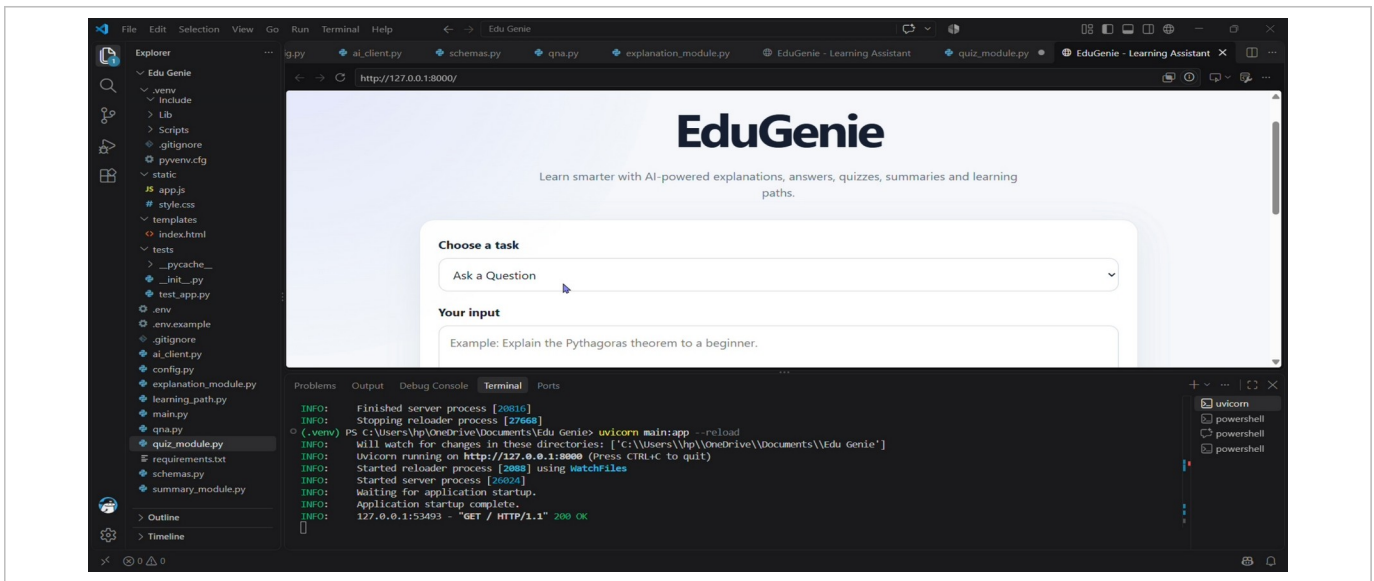


Figure 2: EduGenie home page and learning task selection interface.

7. Project Testing Phase

7.1 Testing Objective

Testing was performed to verify that the application:

- Starts correctly.
- Accepts valid requests.
- Rejects invalid input.
- Routes tasks correctly.
- Returns expected responses.
- Generates correctly structured quiz data.

7.2 Testing Framework

The project uses pytest and FastAPI TestClient. The test file is tests/test_app.py.

7.3 Health Test

```
response = client.get("/health")

assert response.status_code == 200
assert response.json()["status"] == "ok"
```

Expected Result: the server returns HTTP status 200 OK and {"status": "ok"}.

7.4 Input Validation Test

```
response = client.post(
    "/api/task",
    json={
        "task": "qa",
```

```
        "text": ""
    }
)

assert response.status_code == 422
```

Expected Result: the server returns 422 Unprocessable Entity because the input text is required.

7.5 Q&A Endpoint Test

The Q&A endpoint is tested using a mocked response. The test verifies status = 200, task = qa, result = "Mock answer". This allows the API route to be tested without depending on a live Gemini response.

7.6 Quiz Endpoint Test

The quiz endpoint is tested using mock quiz data. The test verifies that `len(data["result"]["questions"]) == 3`. Therefore, the application confirms that the generated quiz contains exactly three questions.

7.7 Functional Testing

Test	Expected Result
Ask a question	Educational answer displayed
Explain a topic	Beginner-friendly explanation displayed
Generate quiz	3 questions with 4 options
Summarize content	Concise summary displayed
Learning recommendation	Beginner-to-advanced learning path

7.8 Test Result

The application was successfully run locally through Uvicorn. The terminal showed:

```
Uvicorn running on http://127.0.0.1:8000
Application startup complete.
```

The browser successfully loaded the EduGenie interface. The API request shown during testing returned:

```
POST /api/task HTTP/1.1 200 OK
```

8. Project Documentation Phase

8.1 Documentation Purpose

The project documentation explains the complete development lifecycle of EduGenie, from initial brainstorming to final demonstration. It contains:

- Project objectives.
- Requirements.
- System design.

- Development process.
- Testing process.
- Demonstration.
- Project structure.
- Source code information.

8.2 Technologies Used

Technology	Purpose
Python	Main programming language
FastAPI	Backend API framework
Uvicorn	Local application server
Google Gemini	Generative AI
HTML	Web page structure
CSS	Interface styling
JavaScript	Frontend interaction
Jinja2	HTML templating
Pydantic	Data validation
Pytest	Testing
python-dotenv	Environment configuration

8.3 API Endpoints

The application provides the following learning endpoints:

- POST /qa
- POST /explain
- POST /quiz
- POST /summarize
- POST /learn/recommendations

It also provides GET /health and the main web page GET /.

9. Project Demonstration Phase

9.1 Starting the Application

First, open the project in Visual Studio Code. Activate the virtual environment and run:

```
uvicorn main:app --reload
```

The terminal should show:

Uvicorn running on http://127.0.0.1:8000
Application startup complete.

Then open http://127.0.0.1:8000

9.2 Demonstration of User Interface

The EduGenie home page contains: EduGenie heading, description, task selection dropdown, input area, submit button, result area.

9.3 Demonstration – Asking a Question

Select “Ask a Question”. Enter: “What is artificial intelligence?”. Click the submit button. EduGenie sends the request to the FastAPI backend, which routes it to the Q&A module. The generated response is displayed in the result area.

9.4 Demonstration – Explaining a Topic

Select “Explain a Topic”. Enter: “Why does the sky look white on cloudy days?”. The application sends the topic to the explanation module. The AI generates a beginner-friendly explanation. The demonstrated output explains concepts such as light scattering and larger particles such as water droplets.

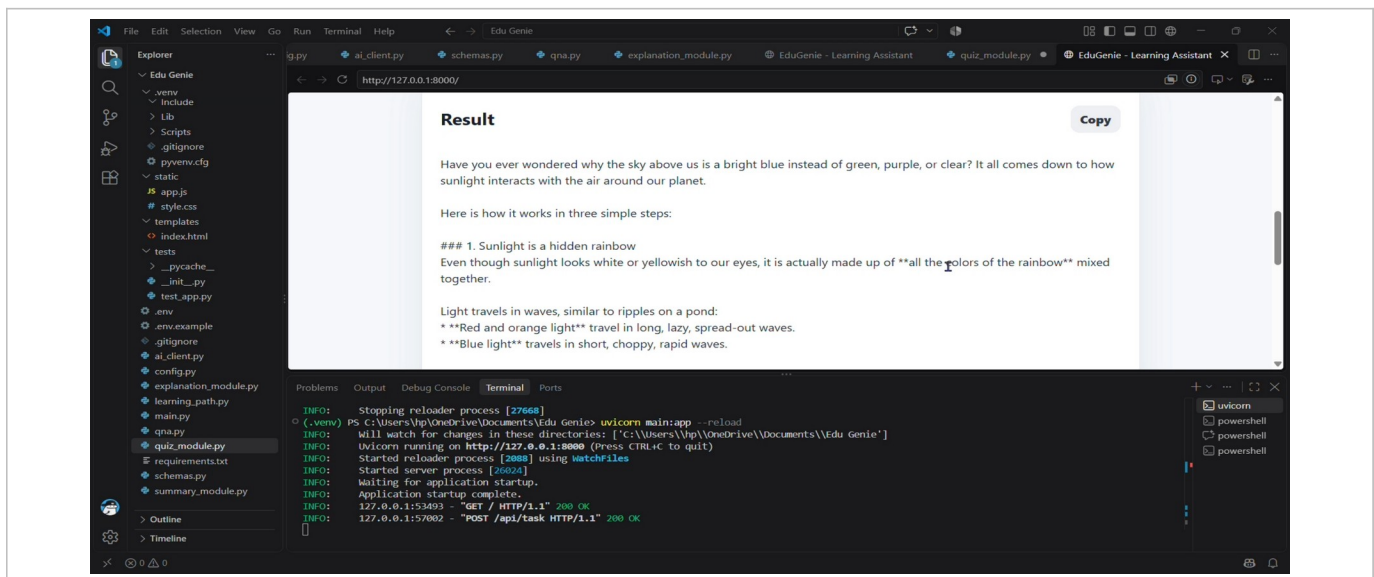


Figure 4a: Selecting “Explain a Concept” and entering a topic.

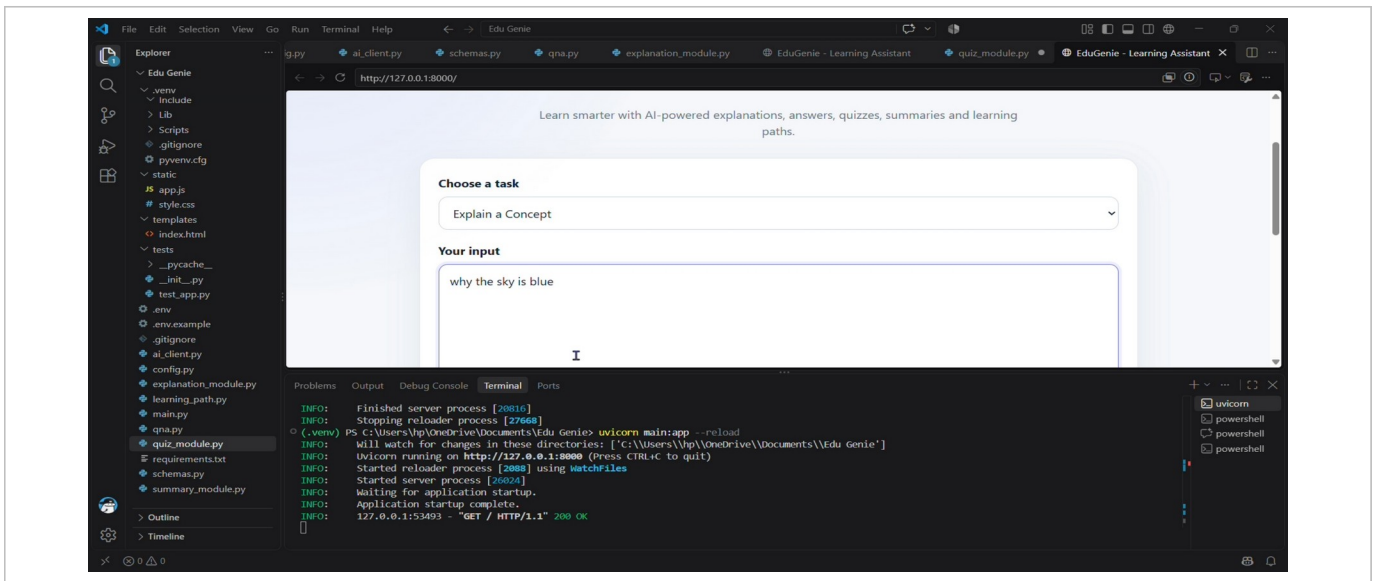


Figure 4b: EduGenie providing a simplified explanation of why the sky is blue.

9.5 Demonstration – Summarization

Select “Summarize”. Paste a long educational paragraph. Click submit. EduGenie processes the passage and generates a shorter version containing the important information.

9.6 Demonstration – Quiz Generation

Select “Generate Quiz”. Enter a topic such as “Basic Science”. The application generates 3 questions, 4 options for every question, correct answer, and explanation.

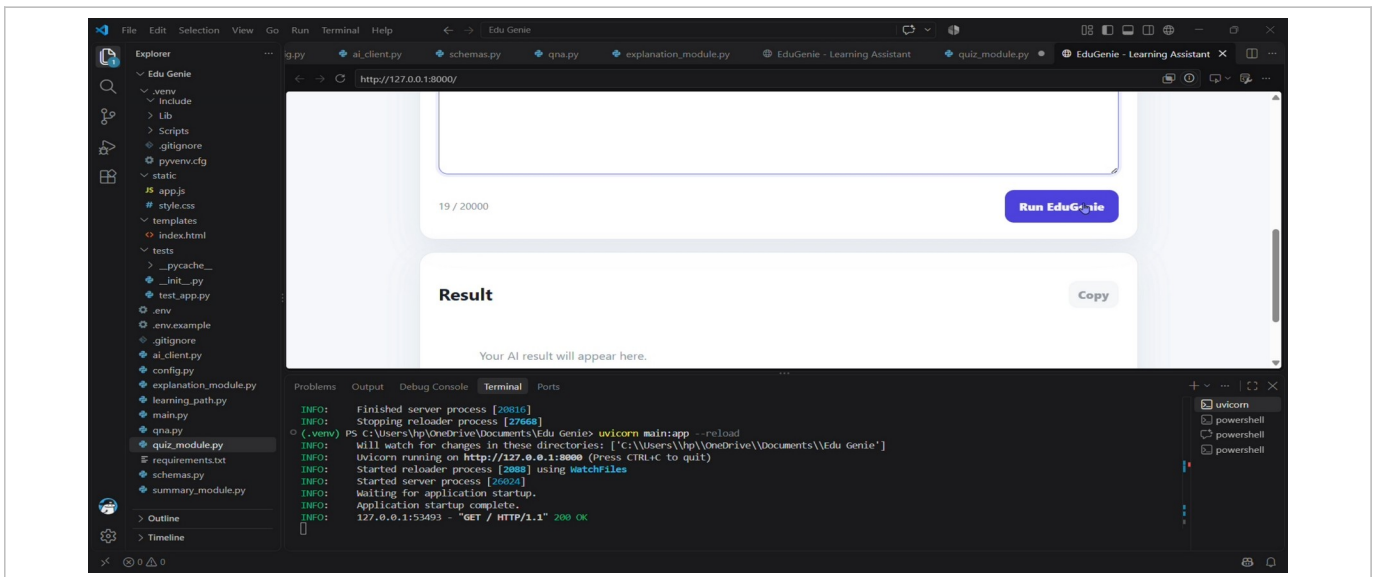


Figure 6: EduGenie task interface awaiting AI-generated result.

The supplied project reference also describes the quiz feature as generating three questions with four options each.

9.7 Demonstration – Learning Recommendations

Select “Learning Recommendations”. Enter: “Python Programming”. The application generates a learning path progressing from beginner concepts towards more advanced topics. The output may include topics, difficulty levels, estimated learning time, objectives, resources and study tips.

10. Conclusion

EduGenie has been developed as an AI-powered educational assistant that combines Generative AI with an intuitive web interface. The application supports:

- Question answering.
- Simplified explanations.
- Quiz generation.
- Summarization.
- Personalized learning recommendations.

The modular architecture allows the different features to be developed and tested independently. FastAPI provides the backend API while HTML, CSS and JavaScript provide the user interface.

The project demonstrates how Generative AI can be integrated into an educational application to provide interactive learning assistance.

11. Future Enhancements

Possible future improvements include:

- Voice-based interaction.
- Multilingual support.
- Mobile application support.
- Student progress tracking.
- Gamification.
- Adaptive learning paths.
- Teacher dashboards.
- Parent dashboards.
- Group study features.
- Image and PDF-based doubt solving.
- Learning Management System integration.
- Smart notifications.

Appendix A – Project Structure

The final project structure is:

```
EduGenie
├── main.py
├── config.py
```

```
├── ai_client.py
├── schemas.py
├── qna.py
├── explanation_module.py
├── quiz_module.py
├── summary_module.py
├── learning_path.py
├── requirements.txt
├── .env
├── .env.example
├── .gitignore
├── templates/
│   └── index.html
├── static/
│   ├── style.css
│   └── app.js
└── tests/
    ├── __init__.py
    └── test_app.py
```

Appendix B – Important Source Code

B.1 main.py

```
from pathlib import Path

from fastapi import FastAPI, Request
from fastapi.responses import HTMLResponse
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates

from schemas import TaskRequest, TaskResponse
from qna import answer_question
from explanation_module import explain_topic
from quiz_module import generate_quiz
from summary_module import summarize_text
from learning_path import get_learning_recommendations

BASE_DIR = Path(__file__).resolve().parent

app = FastAPI(
    title="EduGenie",
    description="Google Gemini Powered Learning Assistant",
    version="1.0.0",
)

app.mount(
    "/static",
    StaticFiles(directory=BASE_DIR / "static"),
    name="static"
)

templates = Jinja2Templates(
    directory=str(BASE_DIR / "templates")
)

@app.get("/", response_class=HTMLResponse)
async def home(request: Request):
    return templates.TemplateResponse(
```

```

        request=request,
        name="index.html",
        context={"title": "EduGenie"},
    )

@app.get("/health")
async def health():
    return {"status": "ok", "service": "EduGenie"}

@app.post("/api/task", response_model=TaskResponse)
async def run_task(payload: TaskRequest):
    task = payload.task
    text = payload.text.strip()

    if task == "qa":
        result = answer_question(text)
    elif task == "explain":
        result = explain_topic(text)
    elif task == "quiz":
        result = generate_quiz(text)
    elif task == "summarize":
        result = summarize_text(text)
    elif task == "learn":
        result = get_learning_recommendations(text)
    else:
        raise ValueError(f"Unsupported task: {task}")

    return TaskResponse(task=task, result=result)

```

B.2 config.py

```

import os
from dotenv import load_dotenv

load_dotenv()

APP_NAME = "EduGenie"

GEMINI_API_KEY = os.getenv("GEMINI_API_KEY", "").strip()
GEMINI_MODEL = os.getenv("GEMINI_MODEL", "gemini-2.5-flash")

USE_LOCAL_EXPLAINER = (
    os.getenv("USE_LOCAL_EXPLAINER", "false").lower() == "true"
)

LOCAL_EXPLAINER_MODEL = os.getenv(
    "LOCAL_EXPLAINER_MODEL", "MBZUAI/LaMini-Flan-T5-783M"
)

MAX_INPUT_CHARS = int(os.getenv("MAX_INPUT_CHARS", "20000"))

```

B.3 explanation_module.py

```

from ai_client import generate_text
from config import USE_LOCAL_EXPLAINER, LOCAL_EXPLAINER_MODEL

def _local_explain(topic: str):
    from transformers import pipeline

```

```

generator = pipeline(
    "text2text-generation",
    model=LOCAL_EXPLAINER_MODEL,
)

prompt = f"""
Explain the following educational topic to a complete beginner.

Use:
- simple language
- short sentences
- one example
- important points

Topic:
{topic}
"""

result = generator(prompt, max_new_tokens=180, do_sample=False)
return result[0]["generated_text"].strip()

def explain_topic(topic: str):
    if USE_LOCAL_EXPLAINER:
        try:
            return _local_explain(topic)
        except Exception:
            pass

    prompt = f"""
Explain the following concept to a beginner.

Topic:
{topic}

Requirements:

- Use plain language.
- Assume the learner has little prior knowledge.
- Give a simple analogy or example.
- Avoid unnecessary jargon.
- Finish with 2 or 3 key takeaways.
"""

    return generate_text(prompt, temperature=0.35, max_output_tokens=2000)

```

B.4 test_app.py

```

from fastapi.testclient import TestClient
import main

client = TestClient(main.app)

def test_health():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json()["status"] == "ok"

```

```

def test_empty_input_rejected():
    response = client.post(
        "/api/task", json={"task": "qa", "text": ""}
    )
    assert response.status_code == 422

def test_qa_endpoint(monkeypatch):
    monkeypatch.setattr(main, "answer_question", lambda text: "Mock answer")
    response = client.post(
        "/qa", json={"task": "qa", "text": "What is Python?"}
    )
    assert response.status_code == 200
    data = response.json()
    assert data["task"] == "qa"
    assert data["result"] == "Mock answer"

def test_quiz_endpoint(monkeypatch):
    mock_quiz = {"questions": [ /* 3 sample MCQ objects */ ]}
    monkeypatch.setattr(main, "generate_quiz", lambda text: mock_quiz)
    response = client.post(
        "/quiz", json={"task": "quiz", "text": "Basic science"}
    )
    assert response.status_code == 200
    data = response.json()
    assert len(data["result"]["questions"]) == 3

```

Final Submission Details

Field	Detail
Project Name	EduGenie – Google Gemini Powered Learning Assistant
Team ID	SWTID-2026-4072
Team Leader	Roshan P
Project Type	Generative AI Web Application
Backend	Python + FastAPI
Frontend	HTML + CSS + JavaScript
AI	Google Gemini
Testing	Pytest + FastAPI TestClient
Application URL (local)	http://127.0.0.1:8000